

Relatório Técnico: Benchmark TPC-H

Comparação de Desempenho: PostgreSQL vs. MongoDB

Kaio da Silva dos Santos

Clara Ribeiro Barreto

Enzo Moraes Chermont Michelotto

Disciplina: Administração de Banco de Dados

Centro Federal de Educação Tecnológica Celso Suckow da Fonseca, CEFET/RJ

Sumário

1	Visão Geral do Trabalho	3
2	Ambiente de Execução	3
3	Dados Utilizados	3
4	PostgreSQL: Passos Realizados	3
4.1	Criação do Banco de Dados	3
4.2	Importação dos Dados	4
4.3	Consultas SQL	4
4.4	Script de Benchmark	4
4.5	Exemplo de Consulta SQL	5
5	MongoDB	5
5.1	Características do MongoDB e Contraste com o PostgreSQL	5
5.2	Modelagem dos Documentos	6
5.3	Importação e Índices	6
5.4	Consultas via Aggregation Pipeline	7
5.5	Script de Benchmark	9
6	Resultados	11
6.1	Análise dos Resultados	11
7	Conclusão	12

1 Visão Geral do Trabalho

Este relatório documenta os passos realizados para comparar o desempenho do PostgreSQL (SGBD relacional) com o MongoDB (SGBD NoSQL orientado a documentos), utilizando as 22 consultas do benchmark TPC-H como base de comparação.

Todo o código-fonte, scripts de importação e consultas utilizados neste trabalho estão disponíveis no repositório do grupo: **Acesse o repositório do projeto.**

2 Ambiente de Execução

Os experimentos foram executados na seguinte configuração de hardware e software:

Tabela 1: Configuração do ambiente

Componente	Especificação
Processador	Intel Core i7-7500U @ 2.70GHz
Memória RAM	16 GB
Armazenamento	SSD Kingston SA400 480GB
Sistema Operacional	Windows 11 64-bit
PostgreSQL	Versão 18.3
MongoDB	8.0
Node.js	Versão 22.15.0

3 Dados Utilizados

Os dados foram fornecidos pelo professor da disciplina, gerados pelo *dbgen* do TPC-H. Os mesmos arquivos *.tbl* foram usados como fonte tanto para o PostgreSQL quanto para o MongoDB, garantindo que a comparação partisse do mesmo conjunto de dados.

Tabela 2: Volume de dados importados por tabela

Tabela	Registros
part	20.000
supplier	1.000
partsupp	80.000
customer	15.000
orders	150.000
lineitem	600.572
nation	25
region	5

4 PostgreSQL: Passos Realizados

4.1 Criação do Banco de Dados

O banco foi criado no pgAdmin 4 com o nome `tpch`. Em seguida, foi criado um schema dedicado chamado `tpc-h`, dentro do qual todas as tabelas do benchmark foram definidas:

Código 1: Criação do schema

```
1 CREATE SCHEMA "tpc-h";  
2 SET search_path TO "tpc-h";
```

As oito tabelas (`part`, `supplier`, `partsupp`, `customer`, `orders`, `lineitem`, `nation`, `region`) foram criadas com os tipos de dados adequados ao PostgreSQL, incluindo `TIMESTAMP` para campos de data e `DECIMAL(12,2)` para valores monetários.

4.2 Importação dos Dados

Os arquivos `.tbl` do TPC-H utilizam `|` como delimitador e incluem um `|` extra ao final de cada linha. Para contornar esse comportamento, foi adicionada uma coluna auxiliar temporária (`stub`) em cada tabela durante a importação, conforme o exemplo abaixo:

Código 2: Importação da tabela orders

```
1 ALTER TABLE orders ADD COLUMN stub CHAR(1);  
2  
3 COPY orders(O_ORDERKEY, O_CUSTKEY, O_ORDERSTATUS, O_TOTALPRICE,  
4             O_ORDERDATE, O_ORDERPRIORITY, O_CLERK,  
5             O_SHIPPRIORITY, O_COMMENT, stub)  
6 FROM 'caminho/para/orders.tbl'  
7 WITH (FORMAT csv, DELIMITER '|', QUOTE E'\x01');  
8  
9 ALTER TABLE orders DROP COLUMN stub;
```

Após a importação de todas as tabelas, as chaves estrangeiras foram adicionadas. Todo esse processo está encapsulado no script `criar_banco.sql`, disponível no repositório do grupo.

4.3 Consultas SQL

As 22 consultas foram obtidas dos arquivos `.sql` fornecidos pelo professor. Os arquivos vieram como templates com parâmetros em branco, os quais foram preenchidos com os valores padrão definidos na especificação oficial do TPC-H [1]. A consulta de número 15 exigiu tratamento especial por envolver três comandos sequenciais: `CREATE VIEW`, `SELECT` e `DROP VIEW`.

Os arquivos `.sql` corrigidos estão disponíveis na pasta `queries/` do repositório do grupo.

4.4 Script de Benchmark

O script de benchmark foi implementado em Node.js (arquivo `consultas_tpch.cjs`), utilizando a biblioteca `pg` para conexão com o banco. Para cada consulta, o script:

1. Lê o arquivo `.sql` correspondente da pasta `queries/`.
2. Executa a consulta no PostgreSQL.
3. Mede o tempo de execução com `console.time()`.
4. Exibe uma amostra de até 10 linhas do resultado.

O script pode ser executado das seguintes formas:

Código 3: Modos de uso do script

```
1 node consultas_tpch.cjs 1          # apenas a Query 1
2 node consultas_tpch.cjs 1 5 13     # Queries 1, 5 e 13
3 node consultas_tpch.cjs all        # todas as 22 queries
```

4.5 Exemplo de Consulta SQL

A Query 1 do benchmark, que realiza agregações sobre a tabela `lineitem`, ilustra a clareza e concisão do SQL em comparação ao *Aggregation Pipeline* do MongoDB:

Código 4: Query 1 em SQL

```
1 SELECT l_returnflag, l_linestatus,
2        sum(l_quantity) as sum_qty,
3        sum(l_extendedprice * (1 - l_discount)) as sum_disc_price,
4        count(*) as count_order
5 FROM lineitem
6 WHERE l_shipdate <= date '1998-12-01'
7 GROUP BY l_returnflag, l_linestatus
8 ORDER BY l_returnflag, l_linestatus;
```

5 MongoDB

5.1 Características do MongoDB e Contraste com o PostgreSQL

O contraste entre o modelo relacional (representado aqui pelo PostgreSQL) de SGBD e o modelo orientado a documentos, que aqui é o MongoDB, evidencia paradigmas fundamentais de armazenamento e recuperação de dados. O PostgreSQL fundamenta-se na teoria relacional, impondo um esquema rígido (*Schema-on-write*), forte tipagem e normalização de dados para mitigar redundâncias e garantir a integridade referencial via chaves estrangeiras (*foreign keys*). E para a reconstrução de entidades complexas, o sistema depende extensivamente de operações de junção algébrica (*JOIN*).

Em contrapartida, o MongoDB adota o formato BSON (Binary JSON), permitindo uma estrutura de dados flexível e sem esquema pré-definido (*Schema-on-read*). Esta característica resolve diretamente o problema do *Impedance Mismatch*, que é a desconexão entre a representação de objetos na camada de aplicação e o modelo tabular no banco de dados. Além dessa flexibilidade, um ponto central do MongoDB consiste no conceito de Localidade de Dados (*Data Locality*). Em vez de normalizar as informações em múltiplas tabelas, o modelo de documentos incentiva a desnormalização. Dados que são frequentemente acessados em conjunto são armazenados fisicamente de forma contígua no mesmo documento, reduzindo significativamente a latência em disco durante operações de leitura pesada, como as exigidas pelo benchmark TPC-H.

Além das diferenças de modelagem, a arquitetura subjacente de escalabilidade difere substancialmente. Enquanto o PostgreSQL tradicionalmente escala de forma vertical (demandando hardware mais robusto), o MongoDB foi desenvolvido para escalabilidade horizontal nativa através de *Sharding*, particionando grandes volumes de coleção através de clusters distribuídos de forma transparente para a aplicação.

5.2 Modelagem dos Documentos

O conjunto de dados fornecido pelo utilitário DBGEN consiste em entidades normalizadas, típicas de um Data Warehouse relacional. A importação direta deste esquema para o MongoDB (um mapeamento 1:1) resultaria em um desempenho subótimo, forçando o uso sistemático do operador \$lookup para simular as junções relacionais, abdicando da principal vantagem do SGBD NoSQL.

Desta forma, optou-se por uma estratégia rigorosa de modelagem aninhada (*Embedding*). A alteração arquitetural de maior impacto prático consistiu na fusão das entidades **orders** e **lineitem**. Em vez de manter duas coleções independentes que exigiriam um cruzamento em tempo de execução para cada consulta de vendas, os registros da tabela **lineitem** foram transformados em um *array* de subdocumentos embutidos diretamente dentro do respectivo documento pai na coleção **orders**.

Esta decisão de design elimina a operação de junção mais custosa de todo o modelo do TPC-H. Ao carregar um documento de pedido para a memória RAM, o MongoDB automaticamente carrega todos os seus itens associados numa única operação de disco, maximizando a eficiência de leitura (em razão do *Data Locality*). As demais entidades periféricas, como **customer** e **supplier**, foram mantidas em coleções separadas para servirem como dimensões de apoio.

5.3 Importação e Índices

Devido ao elevado volume de dados inerente ao benchmark, a "ingestão" (leitura e aninhamento) das tabelas .tbl exigiu o desenvolvimento de uma infraestrutura em Node.js focada em eficiência de memória e rede. Para contornar as limitações de alocação da RAM, a leitura dos arquivos ocorreu por meio de fluxos contínuos, processando e despachando as modificações em lotes através da API bulkWrite do MongoDB. O código a seguir ilustra a lógica de embutimento (*embedding*) dos itens em seus respectivos pedidos de forma assíncrona e massiva.

Código 5: Processo de Ingestão e Aninhamento com BulkWrite em JavaScript

```
1 // Logica de leitura em stream para a tabela Lineitem
2 for await (const line of rl) {
3   if (!line.trim()) continue;
4   const v = line.split('|').slice(0, -1);
5
6   // Mapeamento dos tipos exigidos pelo benchmark
7   const l_orderkey = parseInt(v[0], 10);
8   const item = {
9     l_partkey: parseInt(v[1], 10),
10    l_quantity: parseFloat(v[4]),
11    l_extendedprice: parseFloat(v[5]),
12    l_discount: parseFloat(v[6]),
13    l_shipdate: new Date(v[10])
14    // Demais propriedades omitidas para concisao
15  };
16
17 // Estruturacao da operacao de array $push no pacote do lote
18 bulkOps.push({
19   updateOne: {
```

```

20         filter: { _id: l_orderkey },
21         update: { $push: { lineitems: item } }
22     }
23 });
24
25 // Disparo pontual para prevencao de estouro de memoria (Heap)
26 if (bulkOps.length >= 5000) {
27     await ordersCol.bulkWrite(bulkOps);
28     bulkOps = [];
29 }
30
31
32 }

```

Com a estrutura aninhada populada, a indexação tradicional sofreu alterações fundamentais. A ausência de chaves estrangeiras diretas para a entidade de itens exigiu a criação de Multikey Indexes na coleção de pedidos. Estes índices penetram na estrutura do array, instruindo a Árvore-B (*B-Tree*) do SGBD a criar ponteiros individuais para propriedades específicas embutidas, como as datas de envio dos itens.

Paralelamente, para mitigar a degradação de memória em pesquisas de lookups rasos, implementou-se o conceito de *Covered Indexes* (Índices Cobertos).

Código 6: Estratégia de Indexação Otimizada em JavaScript

```

1  const ordersCol = db.collection('orders');
2
3  // 1. Multikey Indexes (Otimiza buscas em propriedades aninhadas)
4  await ordersCol.createIndex({ "lineitems.l_shipdate": 1 });
5  await ordersCol.createIndex({ "lineitems.l_returnflag": 1, "
   lineitems.l_linestatus": 1 });
6
7  // 2. Covered Index (Evita buscar o documento pesado no disco)
8  await ordersCol.createIndex({ o_custkey: 1, o_comment: 1 });
9
10 // 3. Índice B-Tree convencional
11 await customerCol.createIndex({ c_mktsegment: 1 });

```

5.4 Consultas via Aggregation Pipeline

Diferentemente do padrão SQL, que atua como uma linguagem declarativa onde o programador define o resultado desejado e o motor de banco de dados calcula o plano de execução, o MongoDB emprega a *Aggregation Pipeline*. Este framework funciona de maneira procedural, estruturado como um pipeline de fluxo de dados, no qual os documentos fluem sequencialmente por estágios de filtragem (**\$match**), desconstrução de arrays (**\$unwind**), junção (**\$lookup**) e agrupamento/processamento matemático (**\$group**).

A Query 1 do TPC-H (equivalente à apresentada na Seção 4.5) fornece uma clara demonstração dessa arquitetura. Como os cálculos dependem de variáveis internas do array de itens, o estágio inicial **\$unwind** faz a projeção temporária do documento para possibilitar sumarizações exatas em nível de item. E operações aritméticas complexas são expressas por meio de sintaxe BSON aninhada (ex.: **\$multiply** e **\$subtract**).

Código 7: Pipeline de Agregação - Query 1 em JavaScript

```

1 return await ordersCol.aggregate([
2 // Estagio 1: Desmembra o array em documentos de passagem
   individuais
3 { $unwind: "$lineitems" },
4
5 // Estagio 2: Filtra utilizando o Multikey Index precocemente
6 { $match: { "lineitems.l_shipdate": { $lte: new Date
   ('1998-12-01') } } },
7
8 // Estagio 3: Agrupamento em memoria RAM e matematica acumulativa
9 {
10   $group: {
11     _id: {
12       returnflag: "$lineitems.l_returnflag",
13       linestatus: "$lineitems.l_linestatus"
14     },
15     sum_qty: { $sum: "$lineitems.l_quantity" },
16     // Simulacao de: sum(extendedprice * (1 - discount))
17     sum_disc_price: {
18       $sum: {
19         $multiply: [
20           "$lineitems.l_extendedprice",
21           { $subtract: [1, "$lineitems.l_discount"] }
22         ]
23       }
24     },
25     count_order: { $sum: 1 }
26   }
27 },
28 // Estagio 4: Ordenacao final
29 { $sort: { "_id.returnflag": 1, "_id.linestatus": 1 } }
30
31 ]).toArray();
32

```

Para consultas mais granulares, a pipeline foi otimizada visando impedir transbordamentos de memória. A Query 13, que cruza clientes e pedidos visando aferir frequências logísticas, resultaria numa pane sistêmica caso o array de itens completo fosse carregado no **\$lookup** apenas para filtrar um campo textual. A solução projetada combinou a instrução intra-pipeline **\$project** (para descarte antecipado de memória) com o uso da instrução **\$count**, consolidando o agrupamento inteiramente na camada do servidor de banco de dados.

Código 8: Pipeline de Agregação - Query 13 Otimizada em JavaScript

```

1 async () => {
2 return await customerCol.aggregate([
3   {$lookup: {
4     from: "orders",
5     localField: "c_custkey",
6     foreignField: "o_custkey",

```



```

7       pipeline: [
8         { $project: { o_comment: 1, _id: 0 } }, // project
          garante o uso do índice coberto
9         { $match: { o_comment: { $not: /special.*requests/ } } }
10      ],
11      as: "o"
12  }},
13  { $project: { c_count: { $size: "$o" } } },
14  { $group: { _id: "$c_count", custdist: { $sum: 1 } } },
15  { $sort: { custdist: -1, _id: -1 } }
16  ]).toArray();

```

Através destas adequações procedurais estritas, o modelo orientado a documentos deve provar sua viabilidade frente ao modelo relacional, evidenciando que a chave para a performance em NoSQL reside na união entre design desnormalizado, índices penetrantes e um planejamento do fluxo de dados na memória do SGBD.

5.5 Script de Benchmark

Diferentemente do SGBD relacional, para o qual existem ferramentas padrão para execução de queries via terminal (como o `psql`), a avaliação de performance no MongoDB exigiu o desenvolvimento de um motor de execução customizado. O script de benchmark foi implementado em Node.js, utilizando a biblioteca oficial `mongodb` para conexão com o banco.

A orquestração do benchmark opera através de uma Interface de Linha de Comando (CLI), capturando argumentos passados no terminal via **`process.argv`**. Dessa forma, da mesma forma que o script de benchmark para o PostgreSQL, os testes podem ser executados com granularidade, isolando uma consulta específica ou rodando o conjunto completo:

Código 9: Modos de uso do script

```

1 node consultas_tpch.cjs 1          # apenas a Query 1
2 node consultas_tpch.cjs 1 5 13    # Queries 1, 5 e 13
3 node consultas_tpch.cjs all       # todas as 22 queries

```

Para cada consulta requisitada, o script realiza as seguintes operações:

1. Recupera a *Aggregation Pipeline* correspondente a partir de um dicionário de funções assíncronas.
2. Inicia um cronômetro de alta precisão nativo do Node.js utilizando o método **`console.time()`**.
3. Submete a consulta ao MongoDB e aguarda a devolução completa do cursor de resultados para a memória.
4. Encerra o cronômetro com **`console.timeEnd()`**, registrando o tempo exato decorrido.
5. Processa o retorno aplicando um filtro de sanitização (removendo estruturas binárias BSON pesadas, como os *buffers* de instâncias `ObjectID`) e exibe uma amostra de

até um número 10 (que é alterado conforme desejado) linhas em formato tabular via **console.table()**, simulando o retorno de um grid relacional.

Código 10: Trecho do Código responsável pelas consultas e benchmark em Node.js

```
1 // Laco de execucao das queries passadas no terminal
2 for (const numQuery of QUERIES_A_EXECUTAR) {
3     if (queries[numQuery]) {
4         console.log(\n Iniciando QUERY ${numQuery}...);
5
6         // Marcador o para inicio da medicao
7         console.time('Tempo Q${numQuery}');
8
9         const resultados = await queries[numQuery]();
10
11         // Encerramento imediato do cronometro
12         console.timeEnd('Tempo Q${numQuery}');
13
14         if (MOSTRAR_RESULTADOS) {
15             // Limpeza de artefatos binarios internos do BSON
16             const tabelaFormatada = resultados.slice(0, 10).map(
17                 row => {
18                     const newRow = { ...row };
19                     if (newRow._id === null)
20                         delete newRow._id;
21                     else if (newRow._id && (newRow._id._bsontype ===
22                         'ObjectID' || newRow._id instanceof Buffer)) {
23                         delete newRow._id;
24                     }
25                     return newRow;
26                 });
27             console.table(tabelaFormatada);
28         }
29     }
30 }
```

6 Resultados

Tabela 3: Tempos de execução das 22 consultas TPC-H

Query	PostgreSQL (ms)	MongoDB (ms)
Q1	1657	10634
Q2	135	506
Q3	894	18097
Q4	422	152
Q5	216	26268
Q6	239	4077
Q7	188	153507
Q8	232	45587
Q9	516	195380
Q10	313	5196
Q11	139	67952
Q12	723	4019
Q13	464	15957
Q14	272	5252
Q15	602	3953
Q16	313	28614
Q17	142513	82
Q18	1073	1873
Q19	315	8844
Q20	733733	1141
Q21	438	83980
Q22	197	921

6.1 Análise dos Resultados

De acordo com a Tabela 3, o PostgreSQL foi bem mais rápido na maioria das consultas (19 de 22). Essa vantagem faz sentido, já que o TPC-H foi criado para testar bancos relacionais e algoritmos de junção. No MongoDB, usar o Pipeline de Agregação para simular essas junções é mais complexo e menos eficiente quando os dados estão normalizados. Mesmo assim, o MongoDB se destacou nas consultas Q4, Q17 e Q20. Na Q4, ele ganhou porque evitou o cruzamento de dados. Como os itens já ficam dentro do pedido, ele filtrou tudo diretamente na memória. Já o SQL precisou fazer uma busca externa usando o comando `EXISTS`. Nas consultas Q17 e Q20, a diferença foi enorme: a Q20 levou 12 minutos no PostgreSQL e apenas 1,1 segundo no MongoDB. Para entender esse problema (já que 12 minutos no PostgreSQL indica falha no otimizador ou falta de ajuste de memória, fizemos um teste. Reescrevemos a Q17 e a Q20 usando `JOINS` explícitos no PostgreSQL. Com isso, o tempo caiu para menos de um segundo (636 ms e 779 ms). Mas claro, o TPC-H usa subconsultas de propósito para testar se o Otimizador consegue desdobrá-las. A conclusão não é que a consulta é ruim, mas sim que o Otimizador do PostgreSQL falhou ao criar um plano eficiente para o código original.

7 Conclusão

O teste do TPC-H confirmou que o modelo relacional (PostgreSQL) é melhor para análises dinâmicas, mas também trouxe lições importantes sobre arquitetura e otimização.

É importante destacar que a vitória do MongoDB em algumas consultas é uma comparação injusta. Isso não prova que o motor do MongoDB é mais rápido que o do PostgreSQL, mas mostra que a modelagem de documentos leva grande vantagem em consultas específicas. Ao desnormalizar o banco (colocando os itens de linha dentro dos pedidos no MongoDB), o trabalho pesado de juntar tabelas já foi resolvido na hora de inserir os dados.

Resumindo, a escolha do banco de dados depende de como você vai acessar as informações. Se o sistema precisa de muita flexibilidade para cruzar dados separados (o que exige ajustes do administrador para o Otimizador não falhar), o banco relacional continua sendo a melhor opção. Mas, se a aplicação tem uma hierarquia clara, a desnormalização do modelo NoSQL vai trazer grandes ganhos de velocidade em consultas de leitura.

Referências

- [1] Transaction Processing Performance Council (TPC), *TPC-H Benchmark Specification*, Versão 2.6.1, 2010. Disponível em: <https://www.tpc.org/tpch/>.
- [2] The PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*, 2025. Disponível em: <https://www.postgresql.org/docs/>.
- [3] MongoDB, Inc., *MongoDB Manual*, 2025. Disponível em: <https://www.mongodb.com/docs/>.